

NetSDK Programming Manual

(Voiceprint)



V1.0.0

Foreword

Purpose

Welcome to use NetSDK (hereinafter referred to be "SDK") programming manual (hereinafter referred to be "the Manual").

SDK, also known as network device SDK, is a development kit for developer to develop the interfaces for network communication among surveillance products such as Network Video Recorder (NVR), Network Video Server (NVS), IP camera (IPC), Speed Dome (SD), and intelligence devices.

This document describes the SDK interfaces and calling processes involved in the general business of intelligent video analysis products. For more function modules and structures, refer to the "*NetSDK Development Manual*". For detailed information on basic service processes, including initialization, login, general alarms and intelligent alarms, refer to the *NetSDK Programming Guide*.

The example codes provided in the Manual are only for demonstrating the procedure and not assured to copy for use.

Readers

Software development engineers, product managers and project managers who use SDK.

Safety Instructions

The following symbols might appear in the manual.

Symbol	Description
 Tips	Provides methods to help you solve a problem or save time.
 NOTE	Provides additional information as a supplement to the text.

Revision History

Version	Revision Content	Release Time
V1.0.0	First release	May 2026

About the Manual

- This manual is provided for general reference only. Discrepancies might exist between the product and this manual, including but not limited to printed, electronic, or any other editions.
- The manual will be updated according to the latest laws and regulations of related jurisdictions. Specifications and documentation are subject to change without prior notice. For the latest versions, as well as legal notices, consult the printed manual, the product webpage, our official website, or scan the QR code to access the electronic edition.
- The manual will be updated as necessary to reflect product changes and applicable legal requirements. These updates will not diminish your statutory rights, nor will they remove or weaken the safety information or essential instructions provided at the time of purchase.

- Although we make every effort to ensure accuracy, errors or deviations in descriptions of functions, operations, or technical data might occur. If there is any uncertainty or controversy, we reserve the right of final explanation.
- All trademarks, registered trademarks and company names referenced in this manual are the property of their respective owners.

Glossary

The following explanations of the technical terms used in this document will help you understand the various business functions.

Term	Definition
Main stream	A type of video stream, generally referring to a stream with a large bitrate, low compression ratio, and high image quality, which can provide a better experience when network resources are not restricted.
Sub stream	It has lower resolution and clarity than the main stream, but it uses less network resources; users can choose different stream types according to different applicable scenarios.
Video channel	SDK communicates with devices and represents an abstract concept for video streaming. For example, storage devices (NVRs, etc.) can be connected to several front-end devices (SDs, IPCs, etc.). The storage devices (NVRs, etc.) can abstract the front-end devices (SDs, IPCs, etc.) into video channels for management. If the SDK is directly connected to a single front-end device, the video channel number is typically 0.
Login handle	Maintained internally within the SDK, this unique identifier for communicating with the device is a non-zero integer.
Preview handle	Maintained internally within the SDK, this is a unique identifier for data transmission with a specific video channel of the device, and is a non-zero integer.
Subscription handle	The SDK maintains a unique identifier for subscribing to real-time information reports (smart events, etc.) from devices, which is a non-zero integer.
capability set	A set of capabilities that a device can support for a particular function or set of functions. The capability sets may vary between different equipment models.
Sound anomaly detection	Automatically detects abnormal sound behavior within the channel.

Table of contents

Foreword	II
Glossary.....	IV
Chapter 1 Overview	1
1.1 Introduction.....	1
1.2 Applicability.....	2
1.3 Application scenarios	2
Chapter 2 Function Moduels	3
2.1 SDK initialization.....	3
2.1.1 Introduction.....	3
2.1.2 Interface overview.....	3
2.1.3 Process description	3
2.1.4 Example code	4
2.2 Device login	5
2.2.1 Introduction.....	5
2.2.2 Interface overview.....	5
2.2.3 Process description	5
2.2.4 Example code	7
2.3 Obtain device information	8
2.3.1 Introduction.....	8
2.3.2 Interface overview.....	8
2.3.3 Process description	9
2.3.4 Example code	9
2.4 Intelligent analysis task subscription.....	11
2.4.1 Introduction.....	11
2.4.2 Interface overview.....	11
2.4.3 Process description	12
2.4.4 Example code	13
2.5 Get alarm images.....	16
2.5.1 Introduction.....	16
2.5.2 Interface overview.....	16
2.5.3 Process description	17
2.5.4 Example code	17
2.6 Play alarm audio	26
2.6.1 Introduction.....	26
2.6.2 Interface overview.....	26
2.6.3 Process description	26
2.6.4 Example code	27
2.7 Download alarm audio and video.....	28
2.7.1 Introduction.....	28
2.7.2 Interface overview.....	29
2.7.3 Process description	29
2.7.4 Example code	30
Chapter 3 Interface Definition	31

3.1 SDK initialization.....	31
3.1.1 CLIENT_Init	31
3.1.2 CLIENT_Cleanup	31
3.1.3 CLIENT_SetAutoReconnect.....	31
3.1.4 CLIENT_SetNetworkParam.....	32
3.2 Device login	32
3.2.1 CLIENT_LoginWithHighLevelSecurity.....	32
3.2.2 CLIENT_Logout.....	33
3.3 Obtain device information	34
3.3.1 CLIENT_GetDeviceInfo	34
3.3.2 CLIENT_GetChannelInfo.....	34
3.4 Intelligent analysis task result subscription	35
3.4.1 CLIENT_AttachAnalyseTaskResult.....	35
3.4.2 CLIENT_DetachAnalyseTaskResult.....	35
3.5 Smart event subscription	36
3.5.1 CLIENT_RealLoadPictureEx	36
3.5.2 CLIENT_StopLoadPic	37
3.6 Get event details.....	37
3.7 Download file.....	38
Chapter 4 Callback Functions.....	39
4.1 fSearchDevicesCB	39
4.2 fSearchDevicesCBEx	39
4.3 fDisconnect	39
4.4 fHaveReConnect	40
4.5 fAnalyseTaskResultCallBack.....	40
4.6 Smart event callback fAnalyzerDataCallBack	41
Appendix 1 Security Recommendations	42

Chapter 1 Overview

1.1 Introduction

This document mainly introduces SDK interface reference information, including main function modules, interface functions, and callback functions.

The main functions include: SDK initialization, device initialization, device login, live view, intelligent event reporting and screenshot, and passenger flow statistics.

The files included in the development package will vary depending on the environment as shown below.

- Refer to Table 1-1 for the files included in the Windows Development Kit.

Table 1-1 The files included in Windows development kit

Library type	Library file name	Library file description
Function library	dhnetsdk.h	header file
	dhnetsdk.lib	Lib file
	dhnetsdk.dll	library files
	avnetsdk.dll	library files
Configuration library	avglobal.h	header file
	dhconfigsdk.h	configuration header file
	dhconfigsdk.lib	Lib file
	dhconfigsdk.dll	library files
Playback (encoding/decoding) auxiliary library	dhplay.dll	Playback Library
dhnetsdk auxiliary library	lvsDrawer.dll	Image display library
	StreamConvertor.dll	transcoding library

- Refer to Table 1-2 for a list of files included in the Linux development kit.

Table 1-2 The files included in Linux development kit

Library type	Library file name	Library file description
Function library	dhnetsdk.h	header file
	libdhnetsdk.so	library files
	libavnetsdk.so	library files
Configuration library	avglobal.h	header file
	dhconfigsdk.h	configuration header file
	libdhconfigsdk.so	configuration library
Auxiliary library of libdhnetsdk.so	libStreamConvertor.so	transcoding library



NOTE

- The SDK function library and configuration library are essential libraries.
- The function library is the core of the device network SDK. It is mainly used for communication and interaction between network clients and various products, and is responsible for remote

control, query, configuration, and acquisition and processing of bitstream data.

- The configuration library packages and parses the structures for configuration functions.
- It is recommended to use a playback library for bitstream parsing and playback.
- The auxiliary library is used for audio and video stream decoding for functions such as preview, playback, and intercom, as well as local audio acquisition.

1.2 Applicability

- Recommended memory: at least 512MB.
- The SDK supports the following systems:
 - ◇ Windows
Windows 10/Windows 8.1/Windows 7 and Windows Server 2008/2003.
 - ◇ Linux
Common Linux systems such as Red Hat/SUSE.

1.3 Application scenarios

The SDK can be used in the following scenarios in intelligent analysis devices:

- Commonly used functions include login, preview, intercom, and alarm.
- As a client, it can subscribe to intelligent events from intelligent analysis devices and handle intelligent event reporting; it also can subscribe to intelligent analysis task results from intelligent analysis devices and complete the reporting of analysis result information.

Chapter 2 Function Moduels

2.1 SDK initialization

2.1.1 Introduction

Initialization is the first step for the SDK to perform various business operations. The initialization process itself does not include monitoring functions, but it will set some parameters that affect the overall business.

- SDK initialization will consume some memory.
- Within the same process, only the first initialization is effective.
- After use, call the CLIENT_Cleanup function to release resources.

2.1.2 Interface overview

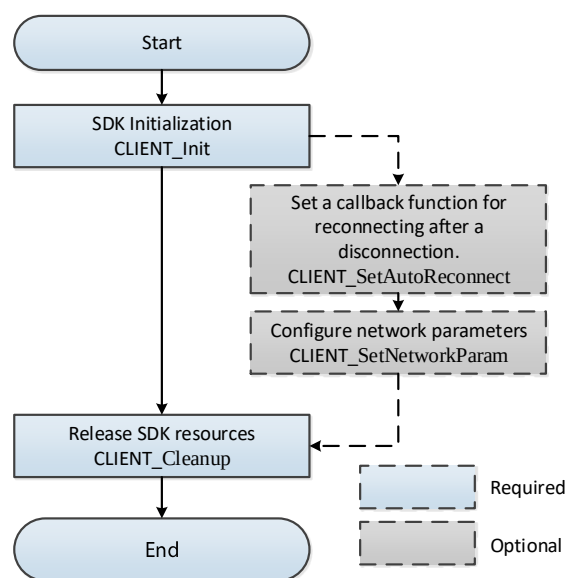
Table 2-1 SDK initialization interface information

interface	Description
CLIENT_Init	SDK initialization interface
CLIENT_Cleanup	SDK cleanup interface
CLIENT_SetAutoReconnect	Configure a callback interface for reconnecting after a disconnection.
CLIENT_SetNetworkParam	Configure network environment interface

2.1.3 Process description

The SDK initialization process is shown in Figure 2-1.

Figure 2-1 SDK Initialization process



S

Process

- Step 1 Call CLIENT_Init to initialize SDK.
- Step 2 (Optional) Call CLIENT_SetAutoReconnect to set the disconnection reconnection callback function. After setting, the SDK will automatically reconnect after a disconnection.
- Step 3 (Optional) Call CLIENT_SetNetworkParam to set network login parameters, which include login device timeout and number of attempts.
- Step 4 After using all SDK functions, call CLIENT_Cleanup to release SDK resources.

Precautions

- The SDK's CLIENT_Init and CLIENT_Cleanup interfaces need to be called in pairs. Multiple pair calls are supported in a single thread, but it is recommended to call them globally only once.
- Initialization: When the CLIENT_Init interface is called multiple times internally, it is only used for internal counting and will not repeatedly request resources.
- Cleanup: The CLIENT_Cleanup interface will clean up all enabled services, such as login, real-time preview, and alarm subscriptions.
- Reconnection: The SDK can be configured to reconnect after a disconnection. When the device disconnects due to special circumstances (such as network outage, power outage, etc.), the SDK will continuously perform login operations at regular intervals until the device is successfully logged in. After a disconnection and reconnection, real-time preview, alarms, and smart image subscription services can be restored, but other services cannot be restored.

2.1.4 Example code

```
// This callback function is set via CLIENT_Init. When the device disconnects, the SDK uses this function to
// notify the user.
void CALLBACK DisConnectFunc(LLONG ILoginID, char *pchDVRIP, LONG nDVRPort, DWORD dwUser)
{
    printf("Call DisConnectFunc: ILoginID[0x%x]\n", ILoginID);
}
// Initialize SDK.
CLIENT_Init(DisConnectFunc, 0);

// .... Calling the functional interface to process business

// Clean up SDK resources
CLIENT_Cleanup();
```

2.2 Device login

2.2.1 Introduction

Device login or user authentication is a prerequisite for conducting other business operations.

A unique login ID is generated when a user logs into the device. The SDK interfaces for other functions require the login ID to be passed in in order to execute. The login ID becomes invalid after logging out of the device.

2.2.2 Interface overview

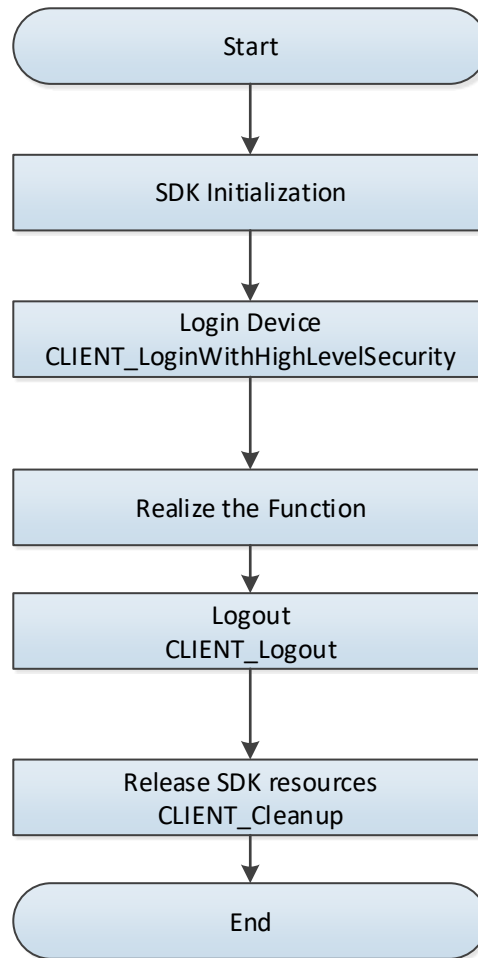
Table 2-2 Device login interface information

interface	Description
CLIENT_LoginWithHighLevelSecurity	High-security login interface.  NOTE CLIENT_LoginEx2 can still be used, but there are security risks. Therefore, it is strongly recommended to use the latest interface CLIENT_LoginWithHighLevelSecurity to log in to the device. The device mentioned in this article generally refers to the audio event detection intelligent server, whose default login port is 37777.
CLIENT_Logout	Logout interface.

2.2.3 Process description

The login process is shown below.

Figure 2-2 Login process



Process

- Step 1 Call CLIENT_Init to initialize SDK.
- Step 2 Call CLIENT_LoginWithHighLevelSecurity to log in to the device.
- Step 3 After successfully logging in, users can access the business functions they need.
- Step 4 After using the function module, call CLIENT_Logout to log out of the device.
- Step 5 After using all SDK functions, call CLIENT_Cleanup to release SDK resource.

Precautions

- Login handle: When login is successful, the interface returns a non-zero value (i.e., the handle may be less than 0, which also indicates successful login); when logging in multiple times on the same device, the login handle will be different each time. Unless there are special business requirements, it is recommended to log in only once. The login handle can be reused for various other business purposes.
- Logout: The interface will release the services that have been opened in the login session, but users are advised not to rely on the cleanup function of the logout interface. For example, after opening the preview, the user should call the interface to end the preview when the preview is no longer needed.
- Login and logout are used in pairs. Login consumes a certain amount of memory and socket information, while logout releases these resources.

- Login failed: It is recommended to perform a preliminary check by checking the error parameter (login error code) of the login interface. Common error codes are shown belowTable 2-3.

Table 2-3 Common error codes

Error code	Description
1	Password incorrect
2	The username does not exist
3	Login timed out
4	Account is logged in
5	The account has been locked
6	The account has been blocked
7	Insufficient resources, busy equipment system
8	Sub connection failed
9	Main connection failed
10	Exceeding the maximum number of user connections
11	Missing avnetsdk or avnetsdk dependency libraries
12	Device not inserted USB flash drive or USB flash drive information error
13	The client IP address does not have login permissions

For more error code information, please refer to the description of the "CLIENT_LoginWithHighLevelSecurity interface" in the "Network SDK Development Manual". The following is an example of code to avoid error code 3:

```
NET_PARAM stuNetParam = {0};
stuNetParam.nWaittime = 8000; // unit ms
CLIENT_SetNetworkParam (&stuNetParam);
```

2.2.4 Example code

```
NET_IN_LOGIN_WITH_HIGHLEVEL_SECURITY stInparam;
memset(&stInparam, 0, sizeof(stInparam));
stInparam.dwSize = sizeof(stInparam);
strncpy(stInparam.szIP, "192.168.1.108", sizeof(stInparam.szIP) - 1);
strncpy(stInparam.szPassword, "123456", sizeof(stInparam.szPassword) - 1);
strncpy(stInparam.szUserName, "admin", sizeof(stInparam.szUserName) - 1);
stInparam.nPort = 37777;
stInparam.emSpecCap = EM_LOGIN_SPEC_CAP_TCP;

NET_OUT_LOGIN_WITH_HIGHLEVEL_SECURITY stOutparam;
memset(&stOutparam, 0, sizeof(stOutparam));
stOutparam.dwSize = sizeof(stOutparam);
LLONG ILoginHandle = CLIENT_LoginWithHighLevelSecurity(&stInparam, &stOutparam);
// Log out of the device
if (0 != ILoginHandle)
{
```

```
CLIENT_Logout(ILLoginHandle);  
}  
  
}
```

2.3 Obtain device information

2.3.1 Introduction

The ASD8000_MIE is a high-performance, multi-functional industrial-grade device (the specific category depends on the actual product definition, such as sensors, controllers, analyzers, etc.), which is widely used in industrial automation, environmental monitoring, energy management and other fields. This product adopts advanced sensing technology and intelligent algorithms, and has high precision, high stability and strong anti-interference capabilities, which can meet the requirements of harsh industrial environments.

When a user accesses the SDK, they can obtain the status and channel information of the added devices from the ASD8000_MIE interface. The device information, including the channel ID, is obtained from this interface. When an alarm occurs later, the channel ID is used to determine which channel triggered the alarm.

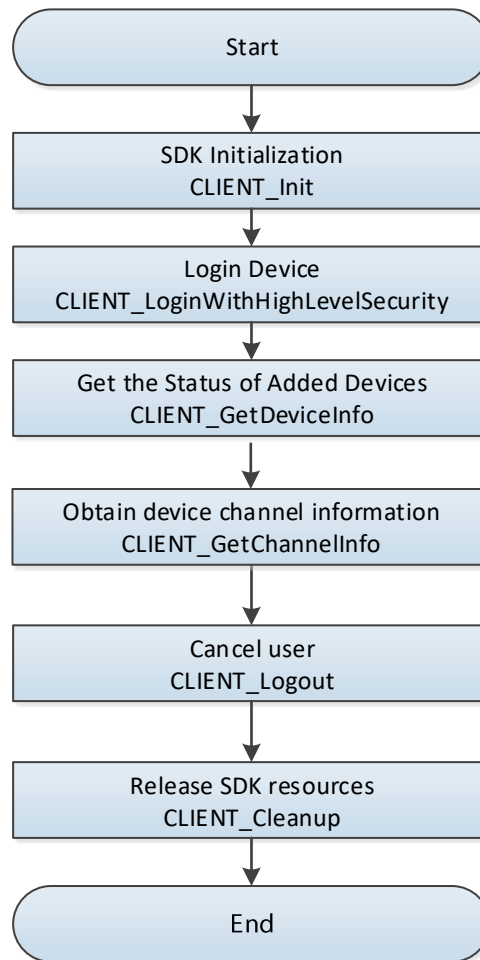
2.3.2 Interface overview

Table 2-4 Get device information interface information

Interface	Description
CLIENT_GetDeviceInfo	Get the status of the added device (ASD8000_MIE)
CLIENT_GetChannelInfo	Obtain device channel information (generally referring to microphones and temperature/vibration sensors connected under ASD8000_MIE).

2.3.3 Process description

Figure 2-3 Get device information process



Process

- Step 1 The CLIENT_Init function is called to complete the SDK initialization process.
- Step 2 After successful initialization, call the CLIENT_LoginWithHighLevelSecurity function to log in to the device.
- Step 3 Call CLIENT_GetDeviceInfo to get the status of the added device.
- Step 4 Call CLIENT_GetChannelInfo to get device channel information.
- Step 5 After using the function module, call CLIENT_Logout to log out of the device.
- Step 6 After using the SDK functions, call the CLIENT_Cleanup function to release SDK resources.

2.3.4 Example code

```
void testGetDevInfo(LLONG ILoginID)
{
    NET_IN_GET_DEVICE_LIST_INFO stuInParam = {sizeof(stuInParam)};

    stuInParam.nCount = 1;
```

```

    strncpy(stuInParam.szDeviceIDs[0], "123456", strlen(stuInParam.szDeviceIDs[0]) - 1);

    NET_OUT_GET_DEVICE_LIST_INFO stOutParam = {sizeof(stOutParam)};

    stOutParam.nMaxCount = 1;

    stOutParam.pstuDeviceInfo = new NET_GET_DEVICE_INFO;

    memset(stOutParam.pstuDeviceInfo, 0, sizeof(NET_GET_DEVICE_INFO));

    BOOL bRet = CLIENT_GetDeviceInfo (ILoginID, &stuInParam, &stOutParam, 5000);

    if (bRet)
    {
        printf("CLIENT_GetDeviceInfo success\n");

        //TODO: Process the returned results
    }
    else
    {
        printf("CLIENT_GetDeviceInfo fail, error:%0x\n", CLIENT_GetLastError());
    }
}

void testGetChanelInfo(LLONG ILoginID)
{
    NET_IN_GET_CHANNEL_INFO stuInParam = {sizeof(stuInParam)};

    strncpy(stuInParam.szDeviceID, "123456", sizeof(stuInParam.szDeviceID)-1);

    NET_OUT_GET_CHANNEL_INFO stOutParam = {sizeof(stOutParam)};

    stOutParam.nMaxCount = 10;

    stOutParam.pstuChannelInfo = new NET_GET_CHANNEL_INFO[10];

    memset(stOutParam.pstuChannelInfo, 0, sizeof(NET_GET_CHANNEL_INFO) * stOutParam.nMaxCount);

    BOOL bRet = CLIENT_GetChannelInfo(ILoginID, &stuInParam, &stOutParam, 5000);

    if (bRet)
    {
        printf("CLIENT_GetChannelInfo success\n");

        //TODO: Process the returned results
    }
    else

```



```
{  
    printf("CLIENT_ CLIENT_GetChannellInfo fail, error:%0x\n", CLIENT_GetLastError());  
}  
}
```

2.4 Intelligent analysis task subscription

2.4.1 Introduction

Users need to upload the algorithm model package and generate an open algorithm through the IVS intelligent server client. The interface for adding new open algorithms allows users to fill in the alarm name, alarm ID, algorithm type, and algorithm target attributes.

After the user sets a custom alarm ID, the alarm will carry an AlarmId field. When connecting to alarms, this field can be used to identify which specific algorithm triggered the alarm. After users integrate the SDK and subscribe to the intelligent analysis results from the device, they can obtain the corresponding intelligent analysis information through the analysis data callback function.

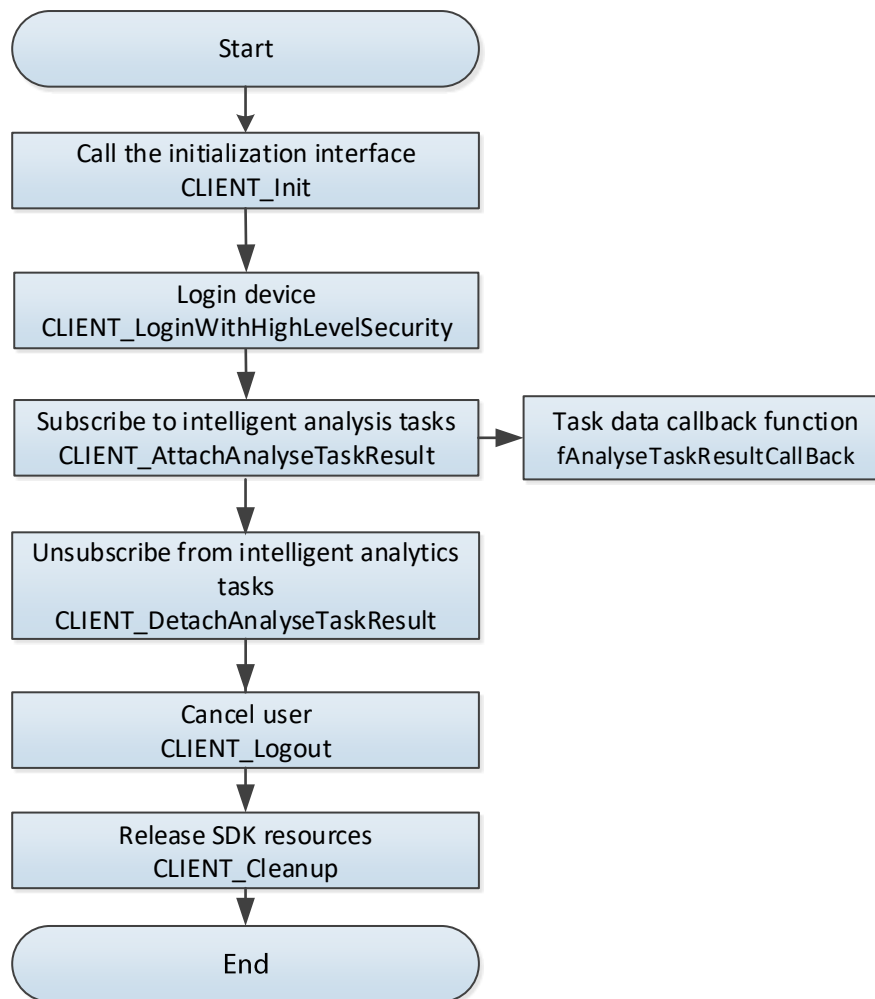
2.4.2 Interface overview

Table 2-5 Interface information

Interface	Description
CLIENT_AttachAnalyseTaskResult	Subscribe to intelligent analytics results
CLIENT_DetachAnalyseTaskResult	Unsubscribe from intelligent analytics results
fAnalyseTaskResultCallBack	Intelligent data analysis callback function

2.4.3 Process description

Figure 2-4 Subscribe to intelligent analysis task process



Process

- Step 1 The CLIENT_Init function is called to complete the SDK initialization process.
- Step 2 After successful initialization, call the CLIENT_LoginWithHighLevelSecurity function to log in to the device.
- Step 3 Call CLIENT_AttachAnalyseTaskResult to start the subscription for the intelligent analysis task (in general, you should avoid having multiple unreleased subscriptions, because each subscription will cause ASD8000_MIE to push an alarm to the subscriber before the subscription is canceled).
- Step 4 The 'fAnalyseTaskResultCallBack' intelligent analysis task callback function is invoked to process the subscribed task data. The extended data in the callback function includes images.
- Step 5 Call CLIENT_DetachAnalyseTaskResult to cancel smart event subscription.
- Step 6 After the using the function module, call the CLIENT_Logout function to exit the device.
- Step 7 After using all SDK functions, call the CLIENT_Cleanup function to release SDK resources.

2.4.4 Example code

```
// Callback function definition (Do not perform time-consuming processing in the callback function; it should
// only be responsible for receiving data. A separate thread should be started to process the data; otherwise, it
// will cause data cache backlog at the ASD8000_MIE sender)

int CALLBACK AnalyseTaskResultCallBack(LLONG IAttachHandle, NET_CB_ANALYSE_TASK_RESULT_INFO
*pstAnalyseTaskResult, const char *pBuf, const DWORD dwBufSize, LDWORD dwUser)
{
    if ((0 == IAttachHandle) || (NULL == pstAnalyseTaskResult))
    {
        return 0;
    }

    char *szState[] = {"Unknown", "Analysis in progress", "Analysis completed", "Analysis failed"};

    for (int i = 0; i < pstAnalyseTaskResult->nTaskResultNum; i++)
    {
        NET_ANALYSE_TASK_RESULT& stuTaskResultInfos = pstAnalyseTaskResult->stuTaskResultInfos[i];

        int nState = (int)stuTaskResultInfos.emFileAnalyseState;

        printf("Information for task %d: task id[%d], file id[%s], analysis status[%s]\n", i+1,
stuTaskResultInfos.nTaskID, stuTaskResultInfos.szFileID, szState[nState+1]);

        for (int j = 0; j < stuTaskResultInfos.nEventCount; j++)
        {
            char szFilePath[256] = {0};

            NET_SECONDARY_ANALYSE_EVENT_INFO& stuEventInfo = stuTaskResultInfos.stuEventInfos[j];

            // Taking open intelligent events as an example
            if (EM_ANALYSE_EVENT_IVS_OPEN_INTELLI == stuEventInfo.emEventType)
            {
                if (stuEventInfo.pstEventInfo)
                {
                    DEV_EVENT_OPEN_INTELLI_INFO *pstInfo =
(DEV_EVENT_OPEN_INTELLI_INFO*)stuEventInfo.pstEventInfo;
```

```

        // Handling sudden changes in sound intensity

        printf("OpenCode:%s,   OpenName:%s,   RuleType:%s\n",   pstInfo->szOpenCode,
pstInfo->szOpenName, pstInfo->szRuleType);

        if (strcmp(pstInfo->szRuleType, "AudioMutation") == 0)
        {
            NET_DEV_EVENT_AUDIO_MUTATION_INFO*   info   =
(NET_DEV_EVENT_AUDIO_MUTATION_INFO*)pstInfo->pstuOpenData;

            printf("event id :%d\n", info->nEventID);

        }

    }

}

else

{

    printf("----Information about event %d: Event type: Unknown \n", j);

}

printf("\n");

}

printf("\n");

}

return 1;

}

static LLONG s_IAttachTaskResult = 0;

void Test_AttachAnalyseTaskResult(LLONG ILoginID)
{
    if (0 != s_IAttachTaskResult)
    {

        // Ensure that the global subscription is only done once to avoid duplicate subscriptions to the same
        ASD8000_MIE.

        BOOL bRet = CLIENT_DetachAnalyseTaskResult(s_IAttachTaskResult);

        if (!bRet)

```

```

    {
        printf("CLIENT_DetachAnalyseTaskResult failed! ErrorCode=%x\n", CLIENT_GetLastError());
        return;
    }

    s_IAttachTaskResult = 0;
}

NET_IN_ATTACH_ANALYSE_RESULT stulnAttach = {sizeof(stulnAttach)};

cout << "Number of tasks to subscribe to (0 means subscribe to all):" << endl;
cin >> stulnAttach.nTaskIdNum;
for (int i = 0; i < stulnAttach.nTaskIdNum; i++)
{
    cout << "Input task" << i+1 << "ID:" << endl;
    cin >> stulnAttach.nTaskIDs[i];
}

stulnAttach.cbAnalyseTaskResult = AnalyseTaskResultCallBack;

cout << "Start listening to intelligent analysis results. Press Esc to stop listening!" << endl;

// Initiate subscription
s_IAttachTaskResult = CLIENT_AttachAnalyseTaskResult(ILLoginID, &stulnAttach, 5000);
if (0 == s_IAttachTaskResult)
{
    printf("Failed to attach analyse task result. ErrorCode=%x\n", CLIENT_GetLastError());
    return;
}

getchar();

```

```
// Unsubscribe, generally called when you no longer need to receive alarms from ASD8000_MIE.  
CLIENT_DetachAnalyseTaskResult(s_IAttachTaskResult);  
  
s_IAttachTaskResult = 0;  
  
cout << "Stop listening to intelligent analysis results!" << endl;  
  
}
```

2.5 Get alarm images

2.5.1 Introduction

After a user subscribes to an alarm event with images, they can obtain the alarm event and alarm image data through the event data callback function.

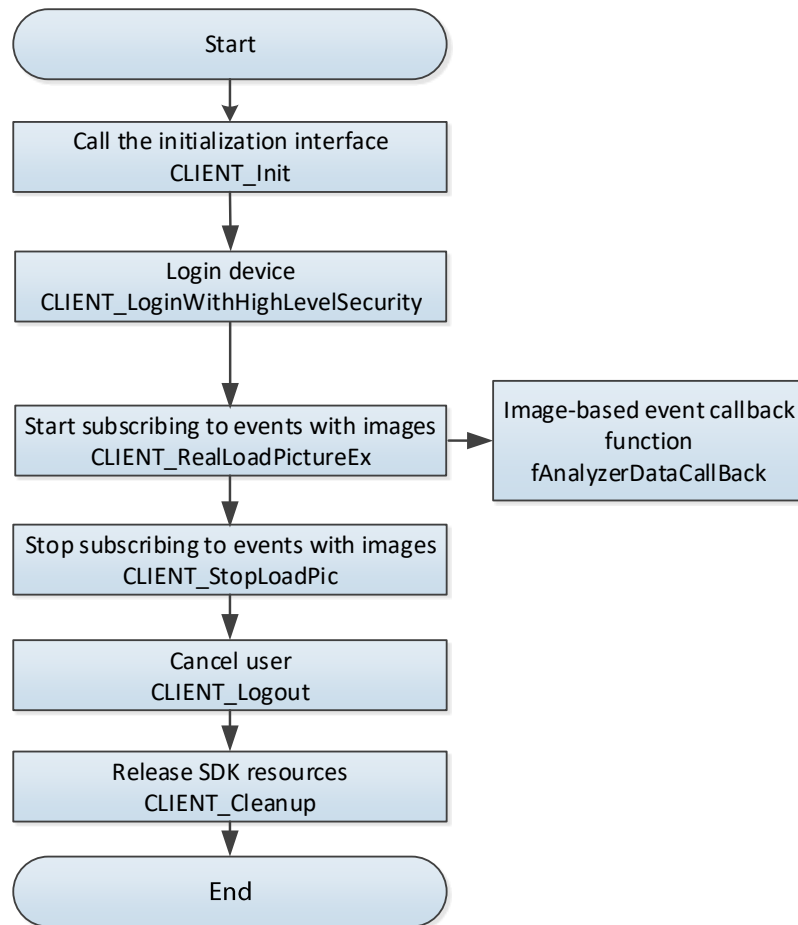
2.5.2 Interface overview

Table 2-6 Information on obtaining alarm images

Interface	Description
CLIENT_RealLoadPictureEx	Start subscribing to events with images
CLIENT_StopLoadPic	Stop subscribing to events with images
fAnalyzerDataCallBack	Event data callback function

2.5.3 Process description

Figure 2-5 Process of obtaining alarm images



Process

- Step 1 Call CLIENT_Init function to complete the SDK initialization process.
- Step 2 After successful initialization, call the CLIENT_LoginWithHighLevelSecurity function to log in to the device.
- Step 3 Call CLIENT_RealLoadPictureEx to start smart event subscription.
- Step 4 The subscribed smart events and image data are processed in the fAnalyzerDataCallBack callback function.
- Step 5 Call the CLIENT_StopLoadPic function to cancel smart event subscription.
- Step 6 After using the function module, call CLIENT_Logout to log out of the device.
- Step 7 After using all SDK functions, call the CLIENT_Cleanup function to release SDK resources.

2.5.4 Example code

```
static BOOL g_bNetSDKInitFlag = FALSE;  
static LLONG g_lLoginHandle = 0L;  
static LLONG g_lRealLoadHandle = 0L;
```

```

static char g_szDevIp[32] = "192.168.4.12";

static int g_nPort = 37777; // TCP connection port must match the TCP port configuration of the device page
you wish to log in to.

static char g_szUserName[64] = "admin";

static char g_szPasswd[64] = "admin";

//*****

// (Collective declaration of) Common callback functions

// Callback function for device disconnection

// It is not recommended to call the SDK interface in this callback function.

// This callback function is set via CLIENT_Init. The SDK will call this function when the device disconnects.

void CALLBACK DisConnectFunc(LLONG ILoginID, char *pchDVRIP, LONG nDVRPort,
DWORD dwUser);

// Callback function for device auto reconnection.

// It is not recommended to call the SDK interface in this callback function.

// This callback function is set via CLIENT_SetAutoReconnect. When a disconnected device successfully
reconnects, the SDK will call this function.

void CALLBACK HaveReConnect(LLONG ILoginID, char *pchDVRIP, LONG nDVRPort,
LDWORD dwUser);

// Smart data analysis callback

// It is not recommended to call the SDK interface in this callback function.

// This callback function is set via CLIENT_RealLoadPictureEx/CLIENT_RealLoadPicture. When a smart image
event is reported from the device, the SDK will call this function.

// nSequence indicates the number of times the same image has been uploaded. 0 indicates the first
occurrence, 2 indicates the last occurrence or only one occurrence, and 1 indicates there are more occurrences
after this one.

// int nState = *(int*) reserved This represents the state of the current callback data. 0 indicates that the current
data is real-time, 1 indicates that the current callback data is offline, and 2 indicates that the offline data
transmission has ended.

// Return value is deprecated and has no special meaning.

int CALLBACK AnalyzerDataCallBack(LLONG IAnalyzerHandle, DWORD dwAlarmType,
void* pAlarmInfo, BYTE *pBuffer, DWORD dwBufSize, LDWORD dwUser, int nSequence, void
*reserved);

//*****

void InitTest()

```



```

{
// Initialize SDK

g_bNetSDKInitFlag = CLIENT_Init(DisConnectFunc, 0);
if (FALSE == g_bNetSDKInitFlag)
{
printf("Initialize client SDK fail;\n");
return;
}
else
{
printf("Initialize client SDK done;\n");
}

// Get SDK version information
// This operation is optional.
DWORD dwNetSdkVersion = CLIENT_GetSDKVersion();
printf("NetSDK version is [%d]\n", dwNetSdkVersion);

// Set up the reconnection callback interface. After setting the callback function for successful reconnection,
the SDK will automatically reconnect when the device disconnects.
// This operation is optional, but it is recommended to set it.
CLIENT_SetAutoReconnect(&HaveReConnect, 0);
// Set login timeout duration and number of attempts.
// This operation is optional.
int nWaitTime = 5000; // Sets the login request response timeout to 5 seconds.
int nTryTimes = 3; // Attempt to establish a connection 3 times during login.
CLIENT_SetConnectTime(nWaitTime, nTryTimes);

// Set more network parameters. The nWaittime and nConnectTryNum members of NET_PARAM have the
same meaning as the login device timeout and number of attempts set by the CLIENT_SetConnectTime
interface.
// This operation is optional.
NET_PARAM stuNetParm = {0};
stuNetParm.nConnectTime = 3000; // The timeout period for trying to establish a link upon login.
CLIENT_SetNetworkParam(&stuNetParm);
NET_IN_LOGIN_WITH_HIGHLEVEL_SECURITY stInparam;

```

```

memset(&stInparam, 0, sizeof(stInparam));

stInparam.dwSize = sizeof(stInparam);

strncpy(stInparam.szIP, g_szDevIp, sizeof(stInparam.szIP) - 1);

strncpy(stInparam.szPassword, g_szPasswd, sizeof(stInparam.szPassword) - 1);

strncpy(stInparam.szUserName, g_szUserName, sizeof(stInparam.szUserName) - 1);

stInparam.nPort = g_nPort;

stInparam.emSpecCap = EM_LOGIN_SPEC_CAP_TCP;

NET_OUT_LOGIN_WITH_HIGHLEVEL_SECURITY stOutparam;

memset(&stOutparam, 0, sizeof(stOutparam));

stOutparam.dwSize = sizeof(stOutparam);

while(0 == g_lLoginHandle)
{
    // Log in to the device

    g_lLoginHandle = CLIENT_LoginWithHighLevelSecurity(&stInparam, &stOutparam);

    if (0 == g_lLoginHandle)
    {
        // The error code can be found in dhnetsdk.h. The output here is in hexadecimal, while the header file outputs
        // decimal. Pay attention to the conversions.

        // For example,

        // #define NET_NOT_SUPPORTED_EC(23) // This feature is not currently supported by the SDK. The
        // corresponding error code is 0x80000017. 23 corresponds to 0x17 in hexadecimal.

        printf("CLIENT_LoginWithHighLevelSecurity %s[%d]Failed!Last Error[%x]\n" ,
            g_szDevIp , g_nPort , CLIENT_GetLastError());
    }
    else
    {
        printf("CLIENT_LoginWithHighLevelSecurity %s[%d] Success\n" , g_szDevIp ,
            g_nPort);
    }

    // When users first log in to the device, the device needs to initialize some data before functions can be
    // realized. We recommend you wait for a while after logging in. The actual waiting time depends on the device.

    Sleep(1000);

    printf("\n");
}

```

```

}

}

void RunTest()
{
if (FALSE == g_bNetSDKInitFlag)
{
return;
}

if (0 == g_ILoginHandle)
{
return;
}

// Subscribe to smart image alerts
LDWORD dwUser = 0;
int nChannel = 0;

// Each time, a corresponding channel is set, and a corresponding event type is assigned.
// To configure this channel to upload all types of events, set the dwAlarmType parameter to EVENT_IVS_ALL.
// If you need to set up a channel to upload two types of events, then call CLIENT_RealLoadPictureEx twice and
pass in different event types.

g_IRealLoadHandle = CLIENT_RealLoadPictureEx(g_ILoginHandle, nChannel, EVENT_IVS_ALL, TRUE,
AnalyzerDataCallBack, dwUser, NULL);

if (0 == g_IRealLoadHandle)
{
printf("CLIENT_RealLoadPictureEx Failed!Last Error[%x]\n",
CLIENT_GetLastError());
return;
}

// Manually capturing an image triggers the intelligent image alarm function
while(1)
{
char szGetBuf[64] = "";
printf("manual snap, \'q\': quit, other: yes\n");

```

```

gets(szGetBuf);

// Entering 'q' exits manual screenshot triggering alarm; other inputs trigger alarms.
if (0 == strncmp(szGetBuf, "q", sizeof(szGetBuf) - 1))
{
break;
}

MANUAL_SNAP_PARAMETER stuSanpParam = {0};
stuSanpParam.nChannel = 0;
memcpy(stuSanpParam.bySequence, "just for test",
sizeof(stuSanpParam.bySequence) - 1);

// Manually capture an image to trigger an alarm function. This function is only valid for ITC devices.
if (FALSE == CLIENT_ControlDeviceEx(g_ILoginHandle, DH_MANUAL_SNAP,
&stuSanpParam))
{
printf("CLIENT_ControlDeviceEx Failed!Last Error[%x]\n",
CLIENT_GetLastError());
break;
}
}
}

void EndTest()
{
printf("input any key to quit!\n");
getchar();

// Stop subscribing to image alerts
if (0 != g_IRealLoadHandle)
{
if (FALSE == CLIENT_StopLoadPic(g_IRealLoadHandle))
{
printf("CLIENT_StopLoadPic Failed!Last Error[%x]\n", CLIENT_GetLastError());
}
}
else

```

```

{
g_lRealLoadHandle = 0;
}
}

// Log out of the device.
if (0 != g_lLoginHandle)
{
if (FALSE == CLIENT_Logout(g_lLoginHandle))
{
printf("CLIENT_Logout Failed!Last Error[%x]\n", CLIENT_GetLastError());
}
else
{
g_lLoginHandle = 0;
}
}

// Clean initialization resources.
if (TRUE == g_bNetSDKInitFlag)
{
CLIENT_Cleanup();
g_bNetSDKInitFlag = FALSE;
}

return;
}

int main()
{
InitTest();
RunTest();
EndTest();

return 0;
}

//*****

```

```

// Common callback set definition

void CALLBACK DisconnectFunc(LLONG ILoginID, char *pchDVRIP, LONG nDVRPort,
DWORD dwUser)
{
printf("Call DisconnectFunc\n");
printf("ILoginID[0x%x]", ILoginID);
if (NULL != pchDVRIP)
{
printf("pchDVRIP[%s]\n", pchDVRIP);
}
printf("nDVRPort[%d]\n", nDVRPort);
printf("dwUser[%p]\n", dwUser);
printf("\n");
}

void CALLBACK HaveReConnect(LLONG ILoginID, char *pchDVRIP, LONG nDVRPort,
LDWORD dwUser)
{
printf("Call HaveReConnect\n");
printf("ILoginID[0x%x]", ILoginID);
if (NULL != pchDVRIP)
{
printf("pchDVRIP[%s]\n", pchDVRIP);
}
printf("nDVRPort[%d]\n", nDVRPort);
printf("dwUser[%p]\n", dwUser);
printf("\n");
}

int CALLBACK AnalyzerDataCallBack(LLONG IAnalyzerHandle, DWORD dwAlarmType, void* pAlarmInfo, BYTE
*pBuffer, DWORD dwBufSize, LDWORD dwUser, int nSequence, void*reserved)
{
if (IAnalyzerHandle != g_IRealLoadHandle)
{

```

```

return 0;

}

int nAlarmChn = 0;

switch(dwAlarmType)
{
case EVENT_IVS_TRAFFIC_OVERLINE:
{
printf("EVENT_IVS_TRAFFIC_OVERLINE event\n");
DEV_EVENT_TRAFFIC_OVERLINE_INFO* pStuInfo =
(DEV_EVENT_TRAFFIC_OVERLINE_INFO*)pAlarmInfo;
nAlarmChn = pStuInfo->nChannelID;
printf("nChannelID[%d]\n", pStuInfo->nChannelID);
}
break;

case EVENT_IVS_PARKINGDETECTION:
{
printf("EVENT_IVS_PARKINGDETECTION event\n");
DEV_EVENT_PARKINGDETECTION_INFO* pStuInfo =
(DEV_EVENT_PARKINGDETECTION_INFO*)pAlarmInfo;
nAlarmChn = pStuInfo->nChannelID;
printf("nChannelID[%d]\n", pStuInfo->nChannelID);
}
break;

default:
printf("other event type[%d]\n", dwAlarmType);
break;
}

if (dwBufSize > 0 && NULL != pBuffer)
{
// Extract the image data pointed to by pBuffer for subsequent processing.
printf("dwBufSize[%d]\n", dwBufSize);
}

```

```

return 1;
}

```

2.6 Play alarm audio

2.6.1 Introduction

Alarm audio can be played in real time as an RTSP stream. After obtaining detailed information about the alarm event, the user can extract the RTSP stream address from the alarm details and then directly request the audio to be played via the RTSP address.

Note: The RTSP server has TLS authentication enabled. Clients must enable TLS when requesting playback.

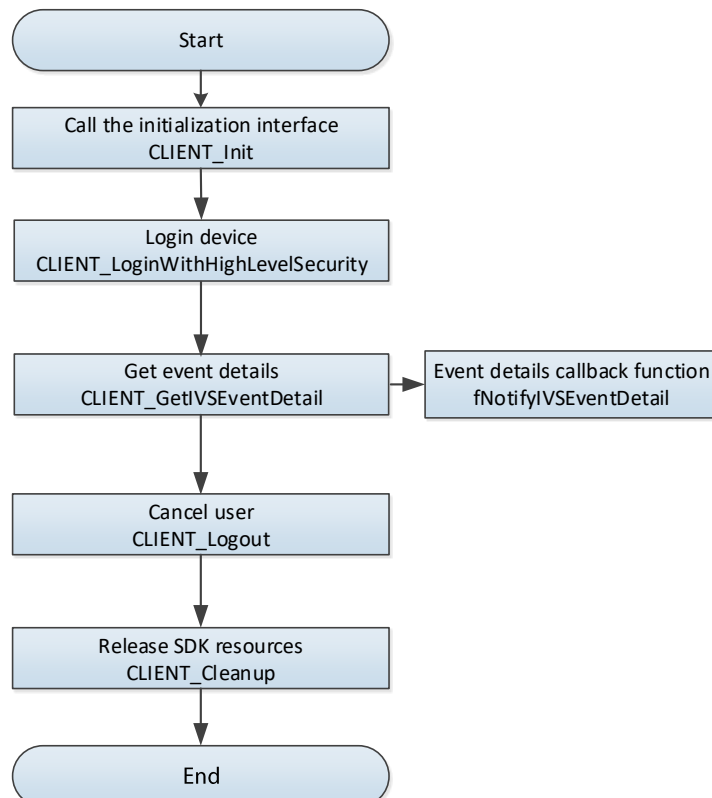
2.6.2 Interface overview

Table 2-7 Get alarm image/audio interface information

Interface	Description
CLIENT_GetIVSEventDetail	Get event details
fNotifyIVSEventDetail	Event details callback

2.6.3 Process description

Figure 2-6 Process of obtaining alarm audio playback address



Process

- Step 1 Call the CLIENT_Init function to complete the SDK initialization process.
- Step 2 After successful initialization, call CLIENT_LoginWithHighLevelSecurity to log in to the device.
- Step 3 Call the CLIENT_GetIVSEventDetail function to obtain detailed event information.
- Step 4 Get event details in the fNotifyIVSEventDetail callback function: audio stream playback address.
- Step 5 After the using the function module, call the CLIENT_Logout function to exit the device.
- Step 6 After using all SDK functions, call the CLIENT_Cleanup function to release SDK resources.

2.6.4 Example code

```
void CALLBACK NotifyIVSEventDetail(NET_NOTIFY_IVSEVENT_DETAIL_INFO* pstuNotifyIVSEventDetailInfo,
LDWORD dwUser)
{
    if(pstuNotifyIVSEventDetailInfo == NULL)
    {
        return;
    }
    printf("pstuNotifyIVSEventDetailInfo->nIVSEventDetailInfoNum:                %d\n",
pstuNotifyIVSEventDetailInfo->nIVSEventDetailInfoNum);
    for(int i = 0; i < pstuNotifyIVSEventDetailInfo->nIVSEventDetailInfoNum; i++)
    {
        printf("-----stuIVSEventDetailInfo[%d]-----\n", i);
        NET_IVSEVENT_DETAIL_INFO&                stuDetail                =
pstuNotifyIVSEventDetailInfo->pstuIVSEventDetailInfo[i];
        printf("nEventInfoNum : %d\n", stuDetail.nEventInfoNum);
        for(int j = 0; j < stuDetail.nEventInfoNum; j++)
        {
            printf("-----stuEventInfo[%d]-----\n", j);
            if(EVENT_IVS_OPEN_INTELLI == stuDetail.stuEventInfo[j].dwAlarmType)
            {
                DEV_EVENT_OPEN_INTELLI_INFO*                pInfo                =
(DEV_EVENT_OPEN_INTELLI_INFO*)stuDetail.stuEventInfo[j].data;
                printf("AnalyzerDataCallBack:EVENT_IVS_OPEN_INTELLI,nChannelID:%d\n",
                    pInfo->nChannelID
                );
                printf("szOpenCode : %s\n", pInfo->szOpenCode);
                printf("szOpenName : %s\n", pInfo->szOpenName);
            }
            else if (EVENT_IVS_TRAFFIC_PARKING == stuDetail.stuEventInfo[j].dwAlarmType)
            {
                DEV_EVENT_TRAFFIC_PARKING_INFO*                pInfo                =
(DEV_EVENT_TRAFFIC_PARKING_INFO*)stuDetail.stuEventInfo[j].data;
            }
        }
    }
}
```

```

    }
    printf("stuDetail.szVideoRtsp : %s\n", stuDetail.szVideoRtsp);
    printf("stuDetail.szVideoDownload : %s\n", stuDetail.szVideoDownload);
    printf("stuDetail.szAudioRtsp : %s\n", stuDetail.szAudioRtsp);
    printf("stuDetail.szAudioDownload : %s\n", stuDetail.szAudioDownload);
    printf("stuDetail.szIotDownload : %s\n", stuDetail.szIotDownload);

    printf("nImageInfoNum : %d\n", stuDetail.nImageInfoNum);
    for(int j = 0; j < stuDetail.nImageInfoNum; j++)
    {
        printf("-----ImageInfo[%d]-----\n", j);
        printf("stuImageInfo[j].emType : %d\n", stuDetail.stuImageInfo[j].emType);
        printf("stuImageInfo[j].szPath : %s\n", stuDetail.stuImageInfo[j].szPath);
    }
}

void Test_Func(LLONG ILoginID, int nChannel, const Json::Value& value)
{
    NET_IN_GET_IVSEVENT_DETAIL_INFO stuInParam = { sizeof(NET_IN_GET_IVSEVENT_DETAIL_INFO) };
    stuInParam.nIdNum = 1;
    stuInParam.nId[0] = 1841088;

    stuInParam.cbNotifyIVSEventDetail = NotifyIVSEventDetail;
    NET_OUT_GET_IVSEVENT_DETAIL_INFO stuOutParam =
{ sizeof(NET_OUT_GET_IVSEVENT_DETAIL_INFO) };
    if (!CLIENT_GetIVSEventDetail(ILoginID, &stuInParam, &stuOutParam, 5000))
    {
        printf("CLIENT_GetIVSEventDetail fail, error:%0x\n", CLIENT_GetLastError());
    }
    else
    {
        printf("CLIENT_GetIVSEventDetail success, error:%0x\n", CLIENT_GetLastError());
    }
}

```

2.7 Download alarm audio and video

2.7.1 Introduction

After obtaining detailed information about the alarm event, users can extract the download links for the audio and video files in the alarm details. After obtaining the address, download the file using the SDK's file download interface.

The default downloaded file is in DAV format. If you need to specify the format, you can append the format field to the download address. For example, appending "format=mp4" (lowercase) means that the downloaded video will be in MP4 format. Currently, DAV and MP4 formats are supported for video, and DAV and WAV formats are supported for audio.

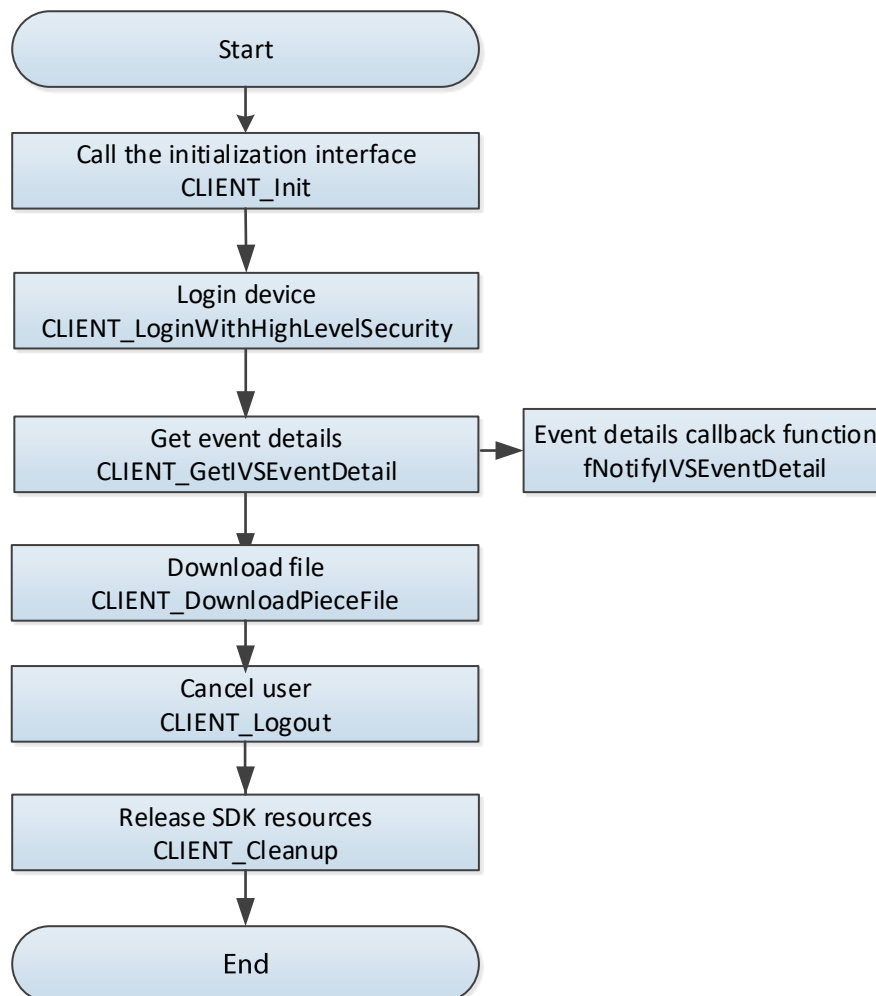
2.7.2 Interface overview

Table 2-8 Get alarm image/audio interface information

Interface	Description
CLIENT_GetIVSEventDetail	Get event details
fNotifyIVSEventDetail	Event details callback
CLIENT_DownloadPieceFile	File download

2.7.3 Process description

Figure 2-7 Downloading alarm audio and video process



Process

- Step 1 Call the CLIENT_Init function to complete the SDK initialization process.
- Step 2 After successful initialization, call the CLIENT_LoginWithHighLevelSecurity to log in to the

device.

Step 3 Call the CLIENT_GetIVSEventDetail function to obtain detailed event information.

Step 4 Get event details in the fNotifyIVSEventDetail callback function.

Step 5 Call CLIENT_DownloadPieceFile to download the file.

Step 6 After using the function module, call the CLIENT_Logout function to exit the device.

Step 7 After using all SDK functions, call the CLIENT_Cleanup function to release SDK resources.

2.7.4 Example code

```
NET_IN_DOWNLOAD_PIECE_FILE stInParam = {sizeof(stInParam)};
strncpy(stInParam.szFileName, "testUrl", sizeof(stInParam.szFileName-1));
stInParam.nOffset = 0; // If the total file length is KB, and each request downloads KB, then nOffset is 0 for the
first request, and 0 for the second request.
stInParam.nNeedLength = 32*1024;
NET_OUT_DOWNLOAD_PIECE_FILE stOutParam = {sizeof(stOutParam)};
stOutParam.szBuffer = new char[50*1024];
stOutParam.nBufferLen = 50*1024;
memset(stOutParam.szBuffer, 0, stOutParam.nBufferLen);

BOOL bRet = CLIENT_DownloadPieceFile(lLoginID, &stInParam, &stOutParam, 5000);
if (bRet)
{
    printf("CLIENT_DownloadPieceFile success\n");
    //Process the returned results
}
else
{
    printf("CLIENT_CLIENT_DownloadPieceFile fail, error:%0x\n", CLIENT_GetLastError());
}
```

Chapter 3 Interface Definition

3.1 SDK initialization

3.1.1 CLIENT_Init

Table 3-1 SDK initialization

Item	Description	
Specification	Initialize the entire SDK	
Function	BOOL CLIENT_Init(fDisconnect cbDisconnect, LDWORD dwUser);	
Parameter	[in]cbDisconnect	Disconnection callback function
	[in]dwUser	User parameters of the disconnection callback function
Return value	Failure: False; Success: True	
Note	- Prerequisites for calling other functions of the network SDK - When the callback function is set to NULL, no callback will be sent to the user after the device disconnects.	

3.1.2 CLIENT_Cleanup

Table 3-2 SDK cleanup CLIENT_Cleanup

Item	Description
Specification	Clean up SDK
Function	void CLIENT_Cleanup()
parameter	None
Return value	None
Note	The SDK cleanup interface is called last before the end.

3.1.3 CLIENT_SetAutoReconnect

Table 3-3 Set the disconnection reconnection callback function

Item	Description	
Specification	Set up an automatic reconnection callback function	
Function	void CLIENT_SetAutoReconnect(fHaveReConnect cbAutoConnect, LDWORD dwUser);	
Parameter	[in]cbAutoConnect	Reconnection callback function
	[in]dwUser	User parameters of the disconnection reconnection callback function

Item	Description
Return value	None
Note	Set up a callback interface for reconnecting after a disconnection. If the callback function is set to NULL, automatic reconnection will not occur.

3.1.4 CLIENT_SetNetworkParam

Table 3-4 Set network parameters

Item	Description
Specification	Configure network environment parameters
Function	void CLIENT_SetNetworkParam(NET_PARAM *pNetParam);
Parameter	[in]pNetParam Parameters such as network latency, number of reconnections, and cache size
Return value	None
Note	The parameters can be adjusted according to the actual network environment.

3.2 Device login

3.2.1 CLIENT_LoginWithHighLevelSecurity

Table 3-5 High-level security login

Item	Description		
Specification	User login device		
Function	LLONG CLIENT_LoginWithHighLevelSecurity (NET_IN_LOGIN_WITH_HIGHLEVEL_SECURITY* pstInParam, NET_OUT_LOGIN_WITH_HIGHLEVEL_SECURITY* pstOutParam);		
Parameter	[in] pstInParam	[in] dwSize	Structure size
		[in] szIP	Device IP
		[in] nPort	Device port
		[in] szUserName	Username
		[in] szPassword	Password
		[in] emSpecCap	Login type
		[in] pCapParam	Login type parameters
	[out] pstOutParam	[in]dwSize	Structure size.
		[out] stuDeviceInfo	Device Information
		[out] nError	Error codes for failure
Return value	Returns the login handle on success, and 0 on failure. After successful login, all operations on the device can be performed through the login handle in conjunction with the SDK interface.		

Item	Description
Note	High-security login interface.  NOTE CLIENT_LoginEx2 can still be used, but there are security risks. Therefore, it is strongly recommended to use the latest interface CLIENT_LoginWithHighLevelSecurity to log in to the device.

For an explanation of the error codes and their meanings for the parameter 'error', please refer to the following table.

Table 3-6 Parameter error code and description

Error code	Description
1	Password incorrect
2	The username does not exist
3	Login timed out
4	Account is logged in
5	The account has been locked
6	The account has been blocked
7	Insufficient resources; busy equipment system
8	Sub connection failed
9	Main connection failed
10	Exceeding the maximum number of user connections
11	Missing avnetsdk or avnetsdk dependency libraries
12	Device not inserted USB flash drive or USB flash drive information error
13	The client IP address does not have login permissions

3.2.2 CLIENT_Logout

Table 3-7 User logout from device

Item	Description
Specification	User logout device
Function	<pre> BOOL CLIENT_Logout(LLONG ILoginID); </pre>
Parameter	[in]ILoginID Return value of CLIENT_LoginWithHighLevelSecurity.
Return value	Failure: False; Success: True
Note	None

3.3 Obtain device information

3.3.1 CLIENT_GetDeviceInfo

Table 3-8 Get the status of added devices

Item	Description	
Specification	Get the status of added devices	
Function	BOOL CLIENT_GetDeviceInfo(LLONG const NET_IN_GET_DEVICE_LIST_INFO* pInParam, NET_OUT_GET_DEVICE_LIST_INFO* pOutParam, int nWaitTime);	
Parameter	[in]ILoginID	Login handle
	[in]pInParam	For interface input parameters, please refer to the definition of the NET_IN_GET_DEVICE_LIST_INFO structure.
	[in]pOutParam	For details on the interface output parameters, please refer to the definition of the NET_OUT_GET_DEVICE_LIST_INFO structure.
	[in]nWaitTime	Interface timeout duration
Return value	Failure: False; Success: True	
Note	None	

3.3.2 CLIENT_GetChannelInfo

Table 3-9 Get device channel information

Item	Description	
Specification	Obtain device channel information	
Function	BOOL CLIENT_GetChannelInfo(LLONG const NET_IN_GET_CHANNEL_INFO* pInParam, NET_OUT_GET_CHANNEL_INFO * pOutParam, int nWaitTime);	
Parameter	[in]ILoginID	Login handle
	[in]pInParam	For interface input parameters, please refer to the definition of the NET_IN_GET_CHANNEL_INFO structure.
	[in]pOutParam	For details on the interface output parameters, please refer to the definition of the NET_OUT_GET_CHANNEL_INFO structure.
	[in]nWaitTime	Interface timeout duration
Return value	Failure: False; Success: True	

Item	Description
Note	None

3.4 Intelligent analysis task result subscription

3.4.1 CLIENT_AttachAnalyseTaskResult

Table 3-10 Start subscribing to the results of the intelligent analytics task

Item	Description	
Specification	Subscribing to the results of intelligent analysis tasks is effective for both "single intelligent analysis tasks" and "intelligent polling analysis tasks".	
Function	<pre>LLONG CALL_METHOD CLIENT_AttachAnalyseTaskResult(LLONG ILoginID, NET_IN_ATTACH_ANALYSE_RESULT *pInParam, int nWaitTime);</pre>	
Parameter	[in] ILoginID	Login handle
	[in] pInParam	Interface input parameters, For details, please refer to the definition of the NET_IN_ATTACH_ANALYSE_RESULT structure.
	[in] nWaitTime	Interface timeout
Return value	- Successfully returned the LLONG type subscription handle - Returns 0 on failure	
Note	If the interface does not respond, use CLIENT_GetLastError to obtain the error code.	

3.4.2 CLIENT_DetachAnalyseTaskResult

Table 3-11 Stop subscribing to the results of the intelligent analytics task

Item	Description	
Specification	Unsubscribe from intelligent analysis task results	
Function	<pre>BOOL CALL_METHOD CLIENT_DetachAnalyseTaskResult(LLONG IAttachHandle);</pre>	
Parameter	[in] IAttachHandle	Subscription handle
Return value	Failure: False; Success: True	
Note	If the interface does not respond, use CLIENT_GetLastError to obtain the error code.	

3.5 Smart event subscription

3.5.1 CLIENT_RealLoadPictureEx

Table 3-12 Description of starting subscription for intelligent events

Item	Description	
Specification	Start subscription for intelligent events	
Function	<pre> LLONG CALL_METHOD CLIENT_RealLoadPictureEx(LLONG ILoginID, int nChannelID, DWORD dwAlarmType, BOOL bNeedPicFile, fAnalyzerDataCallBack cbAnalyzerData, LDWORD dwUser, void* Reserved); </pre>	
Parameter	[in] ILoginID	Login handle
	[in] nChannelID	Channel number of the device.
	[in] dwAlarmType	Type of subscribed alarm event.
	[in] bNeedPicFile	Subscribe to image files or not.
	[in] cbAnalyzerData	Smart event callback function
	[in] dwUser	User-defined data type.
	[in] Reserved	Reserved fields
Return value	- Successfully returned the LLONG type subscription handle - Returns 0 on failure	
Note	If the interface does not respond, use CLIENT_GetLastError to obtain the error code.	

For the intelligent alarm event type, see Table 3-13.

Table 3-13 Description of intelligent event type

dwAlarmType Macro Definition	Value of Macro Definition	Description	Structure Corresponding to Callback pAlarmInfo
EVENT_IVS_ALL	0x00000001	All events.	none
EVENT_IVS_AUDIO_ABNORMALDETECTION	0x00000126	Abnormal audio.	DEV_EVENT_IVS_AUDIO_ABNORMALDETECTION_INFO
EVENT_IVS_OPEN_INTELLI	0x0000039D	Open Intelligent Events	DEV_EVENT_OPEN_INTELLI_INFO
EVENT_IVS_QUANTIFY_MUTATION	0x000004D1	Abnormal mutation alarm events	NET_DEV_EVENT_QUANTIFY_MUTATION_INFO

dwAlarmType Macro Definition	Value of Macro Definition	Description	Structure Corresponding to Callback pAlarmInfo
EVENT_IVS_QUANTIFY_TREND	0x000004D2	Abnormal trend alarm events	NET_DEV_EVENT_QUANTIFY_TREND_INFO

3.5.2 CLIENT_StopLoadPic

Table 3-14 Description of stopping subscription for intelligent events

Item	Description	
Specification	Stop subscribing to intelligent event	
Function	<pre> BOOL CALL_METHOD CLIENT_StopRealPlayEx(LONG IRealHandle); </pre>	
Parameter	[in] IRealHandle	Intelligent event subscription handle.
Return value	BOOL Type Failure: False; Success: True	
Note	If the interface does not respond, use CLIENT_GetLastError to obtain the error code.	

3.6 Get event details

Table 3-15 Get event details

Item	Description	
Specification	Get event details	
Function	<pre> BOOL CALL_METHOD CLIENT_GetIVSEventDetail(LONG ILoginID, const NET_IN_GET_IVSEVENT_DETAIL_INFO* pstuInParam, NET_OUT_GET_IVSEVENT_DETAIL_INFO* pstuOutParam, int nWaitTime); </pre>	
Parameter	[in] ILoginID	Login handle
	[in] pInParam	Interface input parameters, For details, please refer to the following: NET_IN_GET_IVSEVENT_DETAIL_INFO structure definition
	[out] pstuOutParam	Interface output parameters, For details, please refer to the following: NET_OUT_GET_IVSEVENT_DETAIL_INFO structure definition
	[in] nWaitTime	Interface timeout
Return value	Failure: False; Success: True	
Note	If the interface does not respond, use CLIENT_GetLastError to obtain the error code.	

3.7 Download file

Table 3-16 Download file

Item	Description	
Specification	The file download interface can be triggered multiple times to download large files in packets, with each request specifying the filename and download offset.	
Function	<pre> BOOL CALL_METHOD CLIENT_DownloadPieceFile(LLONG ILoginID, const NET_IN_DOWNLOAD_PIECE_FILE* pInParam, NET_OUT_DOWNLOAD_PIECE_FILE* pOutParam, int nWaitTime); </pre>	
Parameter	[in] ILoginID	Login handle
	[in] pInParam	Interface input parameters, For details, please refer to the following: NET_IN_DOWNLOAD_PIECE_FILE structure definition
	[out] pstuOutParam	Interface output parameters, For details, please refer to the following: NET_OUT_DOWNLOAD_PIECE_FILE structure definition
	[in] nWaitTime	Interface timeout
Return value	- Return TRUE for success. - Return FALSE for failure.	
Note	If the interface does not respond, use CLIENT_GetLastError to obtain the error code.	

Chapter 4 Callback Functions

4.1 fSearchDevicesCB

Table 4-1 Search device callback function

Item	Description	
Specification	Search device callback function	
Function	typedef void(CALLBACK *fSearchDevicesCB)(DEVICE_NET_INFO_EX * pDevNetInfo, void* pUserData);	
Parameter	[out]pDevNetInfo	Searched device information
	[out]pUserData	User data
Return value	None	
Note	None	

4.2 fSearchDevicesCBEx

Table 4-2 Asynchronous search device callback function fSearchDevicesCBEx

Item	Description	
Specification	Search device callback function	
Function	typedef void(CALLBACK * fSearchDevicesCBEx)(LLONG ISearchHandle, DEVICE_NET_INFO_EX2 *pDevNetInfo, void* pUserData);	
Parameter	[out] ISearchHandle	Search handle
	[out]pDevNetInfo	Searched device information
	[out]pUserData	User data
Return value	None	
Note	None	

4.3 fDisconnect

Table 4-3 Disconnect callback function

Item	Description
Specification	Disconnection callback function

Item	Description	
Function	<pre>typedef void (CALLBACK *fDisconnect)(LLONG ILoginID, char *pchDVRIP, LONG nDVRPort, LDWORD dwUser);</pre>	
Parameter	[out] ILoginID	Return value of CLIENT_LoginWithHighLevelSecurity.
	[out] pchDVRIP	Disconnected device IP
	[out] nDVRPort	Disconnected device port
	[out] dwUser	User parameters of callback function
Return value	None	
Note	None	

4.4 fHaveReConnect

Table 4-4 Reconnect after disconnection

Item	Description	
Specification	Reconnection callback function	
Function	<pre>typedef void (CALLBACK *fHaveReConnect)(LLONG ILoginID, char *pchDVRIP, LONG nDVRPort, LDWORD dwUser);</pre>	
Parameter	[out] ILoginID	Return value of CLIENT_LoginWithHighLevelSecurity.
	[out] pchDVRIP	Device IP that successfully reconnected after disconnection
	[out] nDVRPort	Device port that successfully reconnected after disconnection
	[out] dwUser	User parameters of callback function
Return value	None	
Note	None	

4.5 fAnalyseTaskResultCallBack

Table 4-5 Intelligent analysis result callback

Item	Description
Specification	Intelligent analysis result callback

Item	Description	
Function	<pre>typedef int (CALLBACK *fAnalyseTaskResultCallBack)(LONG IAttachHandle, NET_CB_ANALYSE_TASK_RESULT_INFO* pstAnalyseTaskResult, const char* pBuf, const DWORD dwBufSize, DWORD dwUser);</pre>	
Parameter	[out]IAttachHandle	CLIENT_AttachAnalyseTaskResult return value
	[out]pstAnalyseTaskResult	Intelligent analysis task result callback information, Refer to the definition of the NET_CB_ANALYSE_TASK_RESULT_INFO structure.
	[out]pBuf	Image cache
	[out]dwBufSize	Image cache size
	[out]dwUser	User data
Return value	None	
Note	None	

4.6 Smart event callback fAnalyzerDataCallBack

Table 4-6 Smart event callback fAnalyzerDataCallBack

Item	Description	
Specification	Smart event callback	
Function	<pre>typedef int (CALLBACK *fAnalyzerDataCallBack)(LONG IAnalyzerHandle, DWORD dwAlarmType, void* pAlarmInfo, BYTE *pBuffer, DWORD dwBufSize, DWORD dwUser, int nSequence, void *reserved);</pre>	
Parameter	[out]IAnalyzerHandle	Return value of CLIENT_RealLoadPictureEx
	[out]dwAlarmType	Intelligent event type
	[out]pAlarmInfo	Event information cache
	[out]pBuffer	Image cache
	[out]dwBufSize	Image cache size
	[out]dwUser	User data
	[out]reserved	reserve
Return value	None	
Note	None	

Appendix 1 Security Recommendations

1. Account Management

1.1 Use Strong Passwords

- The length should not be less than 8 characters.
- Include at least two types of characters; character types include upper and lower case letters, numbers and symbols.
- Do not contain the account name or the account name in reverse order.
- Do not use continuous characters, such as 123, abc, etc.
- Do not use overlapped characters, such as 111, aaa, etc.

1.2 Change Password Regularly

It is suggested that you change passwords regularly to reduce the risk of being guessed or cracked.

1.3 Assign Accounts and Permissions Reasonably

According to business and management needs, reasonably add new users, and reasonably allocate a minimum set of permissions for them.

1.4 Enable Account Lock

The account lock feature is enabled by default, and it is recommended to keep it on to guarantee the account security. If an attacker attempts to log in with the wrong password several times, the corresponding account and the source IP address will be locked.

1.5 Set and Update Passwords Reset Information Timely

The platform supports password reset function. To reduce the risk of being attacked, please set up related information for password reset in time. If the information changes, please modify it in time. When setting password protection questions, it is suggested not to use those that can be easily guessed.

1.6 Enable Account Binding IP/MAC

It is recommended to enable the account binding IP/MAC mechanism to further improve access security.

2. Service Configuration

2.1 Enable HTTPS

It is suggested to enable HTTPS, so that you visit web service through a secure communication channel.

2.2 Disable Unnecessary Services and Choose Secure Modes

If not needed, it is recommended to turn off some services such as SNMP, SMTP, etc., to reduce risks.

If necessary, it is highly recommended that you use safe modes, including but not limited to the following services:

- SMTP: Choose TLS to access mailbox server.
- FTP: Choose SFTP, and set up strong passwords.

3. Network Configuration

3.1 Enable Firewall Allowlist

It is suggested to enable allowlist function to prevent everyone, except those with specified IP addresses, from accessing the system. Therefore, please be sure to add your computer's IP address and the accompanying equipment's IP address to the allowlist.

3.2 Network Isolation

The network should be isolated by partitioning the video monitoring network and the office network on the switch and router to different VLANs. This prevents attackers from using the office network to launch Pivoting attacks on the video monitoring network.

4. Security Auditing

4.1 Check Online Users

It is recommended to check online users irregularly to identify whether there are illegal users logging in.

4.2 View the Platform Log

By viewing the log, you can get the IP information of the attempt to log in to the platform and the key operation information of the logged-in user.

5. Physical Protection

It is suggested to perform physical protection to the device that has installed the platform. For example, place the device in a special computer room and cabinet, and implement well-done access control permission and key management to prevent unauthorized personnel from carrying out physical contacts such as damaging hardware.

6. Perimeter Security

It is suggested to deploy perimeter security products and take necessary measures such as authorized access, access control, and intrusion prevention to protect the software platform security.